Classification Tree and MapReduce: Worked Solutions

Introduction to AI and Big Data Analytics (final assessment, applied questions) | Tien Dat Ly

Two applied problems from the course: evaluating a classification tree for overfitting, and designing a MapReduce word count on two machines. Each step shows the working, not just the answer.

Part 1: Classification tree (hiking decision)

Problem. Predict whether a person goes hiking (Go / Not Go) from three features: weekend, friends, and weather. The tree was built on 8 training rows and evaluated on 5 test rows.

Training data (8 rows)

Weekend?	Friends?	Weather?	Go hiking?
Yes	Yes	Sunny	Go
Yes	No	Rainy	Not Go
No	Yes	Sunny	Go
Yes	No	Sunny	Go
No	Yes	Rainy	Not Go
No	No	Sunny	Not Go
Yes	Yes	Rainy	Go
No	No	Rainy	Not Go

Step 1: Count rows at each leaf and assign the majority class

I traced every training row down the tree and counted Go and Not Go at each leaf. The leaf prediction is the majority class (Go when #Go is at least #Not Go).

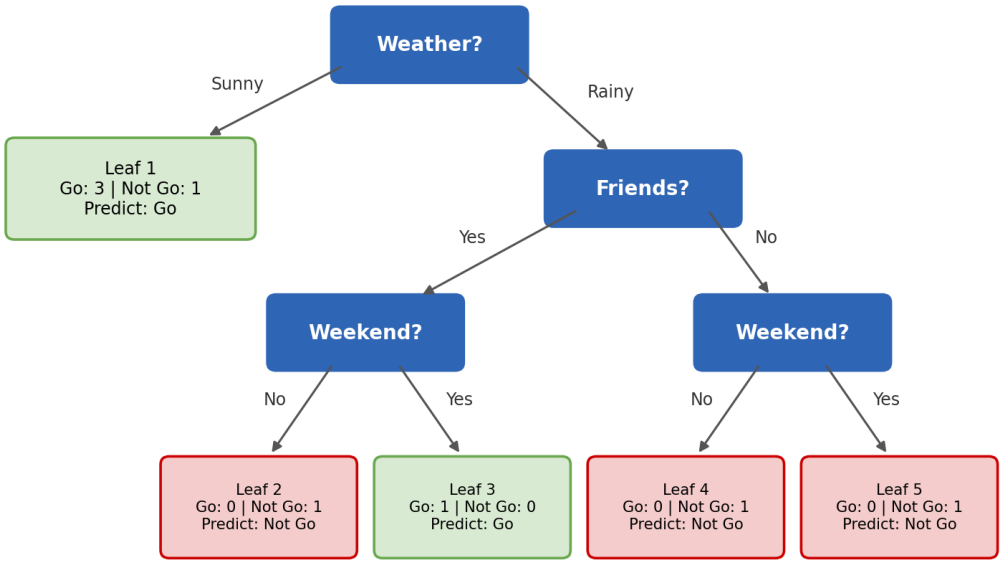


Figure 1: Classification tree with leaf counts and predictions

Leaf	Path	#Go	#Not Go	Prediction
1	Weather = Sunny	3	1	Go
2	Rainy, Friends = Yes, Weekend = No	0	1	Not Go
3	Rainy, Friends = Yes, Weekend = Yes	1	0	Go
4	Rainy, Friends = No, Weekend = No	0	1	Not Go
5	Rainy, Friends = No, Weekend = Yes	0	1	Not Go

Step 2: Evaluate on the training data

Weekend?	Friends?	Weather?	Actual	Predicted	Result
Yes	Yes	Sunny	Go	Go	TP
Yes	No	Rainy	Not Go	Not Go	TN
No	Yes	Sunny	Go	Go	TP
Yes	No	Sunny	Go	Go	TP
No	Yes	Rainy	Not Go	Not Go	TN
No	No	Sunny	Not Go	Go	FP
Yes	Yes	Rainy	Go	Go	TP
No	No	Rainy	Not Go	Not Go	TN

Confusion counts: TP = 4, FP = 1, TN = 3, FN = 0.

Accuracy = (TP + TN) / total rows = (4 + 3) / 8 = 7/8 = 87.5%

Step 3: Evaluate on the test data

Weekend?	Friends?	Weather?	Actual	Path taken	Predicted	Result
No	Yes	Sunny	Go	Sunny (Leaf 1)	Go	TP
Yes	No	Sunny	Go	Sunny (Leaf 1)	Go	TP
Yes	No	Rainy	Go	Rainy, No friends, Weekend (Leaf 5)	Not Go	FN
No	No	Rainy	Not Go	Rainy, No friends, No weekend (Leaf 4)	Not Go	TN
No	Yes	Rainy	Not Go	Rainy, Friends, No weekend (Leaf 2)	Not Go	TN

Confusion counts: TP = 2, FP = 0, TN = 2, FN = 1.

Accuracy = (2 + 2) / 5 = 4/5 = 80.0%

Step 4: Diagnose the model (overfitting) and fix it

Diagnosis: overfitting. The tree grew so deep that Leaves 2, 3, 4 and 5 each hold a single training row. It memorises individual examples and captures noise instead of the general pattern. Accuracy falls from 87.5% on training data to 80.0% on unseen test data, and the model misses a test case (a rainy weekend with no friends, which actually was a Go).

Fix: prune the tree.

- Prune with cross-validation (for example `cv.tree` in R), choosing the subtree size with the lowest cross-validated error.
- Or limit growth up front: set a higher minimum number of rows required to split, or a maximum depth.

Why it works: a smaller tree cannot isolate single rows, so each leaf is supported by more data, which reduces variance and should generalise better to new data.

Step 5: 3-fold cross-validation on the combined data

- Combine training and test data: 8 + 5 = **13 records**.
- Split into 3 non-overlapping folds. 13 does not divide evenly, so the folds have **4, 4 and 5** records.
- Run 3 rounds. In each round, train the tree on 2 folds and test on the remaining fold, recording that round's accuracy.
- The final testing accuracy is the **average of the 3 round accuracies**, so every record is used for testing exactly once.

The exact figure depends on which records fall into each fold, so the method matters more than a single number here.

Part 2: MapReduce word count on two machines

Problem. Count each word across a list of documents (ID, text), ignoring stop words, using only two machines. The aim is to arrange the work so both machines finish at about the same time.

ID	Text
1	Sarah had met James
2	basket of ripe cherries
3	James and Sarah met
4	The basket of the ripe cherries

Step 1: Words to count and expected result

Stop words ignored: had, of, and, the. Words counted: Sarah, met, James, basket, ripe, cherries.

Final result: Sarah 2, met 2, James 2, basket 2, ripe 2, cherries 2

Step 2: Map, shuffle and reduce

- **Map:** each document is processed independently. The text is split into words, stop words are dropped, and for every remaining word the mapper emits (**key** = the word, **value** = 1).
- **Shuffle and sort:** all pairs with the same key are grouped into (key, [1, 1, ...]).
- **Reduce:** the reducer sums the list of 1s for each key and emits (key, total).

Document	Map output (key, value)
Doc 1	(Sarah,1) (met,1) (James,1)
Doc 2	(basket,1) (ripe,1) (cherries,1)
Doc 3	(James,1) (Sarah,1) (met,1)
Doc 4	(basket,1) (ripe,1) (cherries,1)

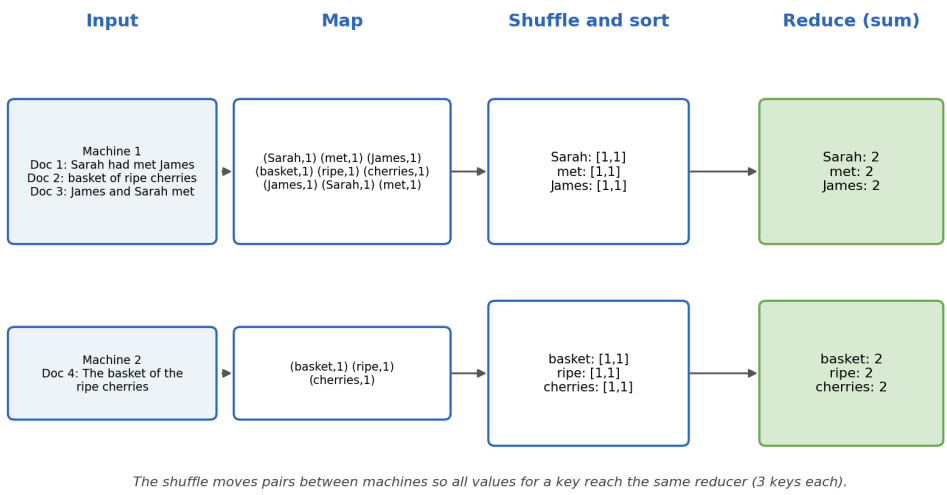


Figure 2: MapReduce data flow for the four documents

Step 3: Balance the reduce stage across the two machines

Machine 1 starts with documents 1 to 3 and Machine 2 with document 4. There are 6 unique keys, and each key ends up with exactly 2 values, so every key costs the same to reduce. To minimise the remaining time, the shuffle should send **3 keys (with their full value lists) to each machine**. For example, Machine 1 reduces Sarah, met and James, and Machine 2 reduces basket, ripe and cherries. This keeps both machines equally busy so neither sits idle.

Step 4: Distribute five documents to minimise total time

A fifth document is added: "Ripe cherries are in the basket" (the mapper lower-cases words so "Ripe" and "ripe" are the same key). The map stage dominates the cost, so I measured the workload as the number of target words each machine has to process.

Document	Target words kept	Count
Doc 1	Sarah, met, James	3
Doc 2	basket, ripe, cherries	3
Doc 3	James, Sarah, met	3
Doc 4	basket, ripe, cherries	3
Doc 5	ripe, cherries, basket	3

Every document has the same workload (3 words), and 5 documents cannot be split evenly, so the best split is 3 documents and 2 documents:

Machine	Documents	Map workload
Machine 1	Docs 1, 2, 3	9 words
Machine 2	Docs 4, 5	6 words

Any other split is worse. For example, 4 documents on one machine and 1 on the other gives 12 words against 3, so one machine would finish long after the other and set the total time. A 3 and 2 split keeps the load as close to 50/50 as the data allows.